Modéliser les besoins client par un diagramme de cas d'utilisation (UML)

Introduction au langage de modélisation UML

- **Définition :**
  UML (Unified Modeling Language) est un langage **visuel** de modélisation, utilisé en ingénierie logicielle pour représenter graphiquement les différents aspects d'un système.
- **But principal :**
  - Faciliter la communication entre les acteurs du projet (client, analystes, développeurs, testeurs).
  - Fournir une vision claire et commune du système à réaliser.
- **Caractéristiques :**
  - UML n'est pas un langage de programmation, mais un **langage de modélisation**.
  - C'est une **norme internationale** (OMG).

OMG veut dire **Object Management Group**. C'est :

Une **organisation internationale à but non lucratif** fondée en 1989.

Elle regroupe des entreprises et experts du domaine informatique.

Son rôle principal est de **définir et standardiser des langages et outils de modélisation**.

Parmi ses standards les plus connus :

- **UML (Unified Modeling Language)** → langage de modélisation.
- **BPMN (Business Process Model and Notation)** → pour modéliser les processus métiers.

Formateur : AARAB HAFID

- **CORBA (Common Object Request Broker Architecture)** → pour l'interopérabilité entre systèmes.

  UML est standardisé par l'OMG , c'est **un langage officiel, reconnu et normalisé au niveau international**.

  - Il contient différents types de diagrammes (13 au total) : structurels (classe, objet, composant…) et comportementaux (cas d'utilisation, séquence, activité…).

- **Pourquoi UML est important ?**
  - Compréhension facile du système, même pour les non-informaticiens.
  - Réduction des ambiguïtés dans les spécifications.
  - Base solide pour la conception et le développement.

Définition du diagramme des cas d'utilisation

- **Définition :**
  Le **diagramme de cas d'utilisation** (Use Case Diagram) est un **diagramme comportemental** d'UML qui décrit **ce que fait le système** du point de vue de l'utilisateur.
- **Rôle :**
  - Représenter les fonctionnalités attendues par le client.
  - Mettre en avant les interactions entre les utilisateurs (acteurs) et le système.
  - Servir de point de départ pour l'analyse et la conception.

- **Remarque :**
  Le diagramme de cas d'utilisation décrit le **QUOI** (les besoins) et non le **COMMENT** (la solution technique).

Les acteurs

- **Définition :**
  Un acteur est une **entité externe au système** qui interagit avec celui-ci.
- **Types d'acteurs :**
  - **Acteur humain** : un utilisateur qui utilise le système (client, employé, administrateur).
  - **Acteur système** : un autre logiciel, une base de données ou un périphérique qui interagit avec le système.
- **Caractéristiques :**
  - Les acteurs représentent des **rôles** et non des personnes spécifiques.
  - Un même utilisateur peut jouer plusieurs rôles (ex. un employé peut aussi être client).

Les cas d'utilisation

- **Définition :**
  Un cas d'utilisation représente une **fonctionnalité spécifique du système**, perçue par l'acteur.
- **Exemples :**
  - Dans un site e-commerce : *S'inscrire*, *Se connecter*, *Commander un produit*, *Payer en ligne*.
  - Dans une application bancaire : *Consulter solde*, *Effectuer un virement*, *Imprimer un relevé*.
- **Caractéristiques :**
  - Chaque cas d'utilisation doit apporter une valeur à l'acteur.
  - Il est représenté par une **ellipse** dans le diagramme UML.

Relations entre acteurs et cas d'utilisation

1. Relation d'association

- Relie un acteur à un cas d'utilisation.
- Signifie que l'acteur **participe** ou **interagit** avec le cas d'utilisation.
- Représentée par une **ligne simple**.

2. Multiplicité

- Un acteur peut être associé à **plusieurs cas d'utilisation**.
- Un cas d'utilisation peut aussi impliquer **plusieurs acteurs**.

  Exemple :

  - L'acteur *Client* peut réaliser : *S'inscrire*, *Commander*, *Payer*.
  - Le cas d'utilisation *Payer* peut concerner *Client* et *Banque*.

Relations entre cas d'utilisation

1. Inclusion (<<include>>)

- Utilisé quand un cas d'utilisation **réutilise systématiquement** un autre.
- Exemple : *Commander un produit* inclut *S'authentifier*.
- Avantage : factoriser les comportements communs.

2. Extension (<<extend>>)

- Un cas d'utilisation **peut être étendu par un autre** si certaines conditions sont remplies.
- Exemple : *Payer une commande* peut être étendu par *Demander un code promotionnel*.
- Avantage : modéliser des comportements optionnels.

3. Point d'extension

- Définit où un cas d'utilisation peut être étendu.

- Exemple : *Payer* → [Point d'extension : mode de paiement].

Relation de généralisation / spécialisation

- **Définition :**
  Comme en programmation orientée objet, un cas d'utilisation ou un acteur peut être **généralisé**.
- **Exemple avec acteurs :**
  - Acteur *Utilisateur* généralisé en *Client* et *Administrateur*.
- **Exemple avec cas d'utilisation :**
  - Cas *Effectuer transaction* généralisé en *Virement* et *Retrait*.
- **Avantage :**
  Réutilisation et organisation plus claire des rôles et fonctionnalités.

Description textuelle des cas d'utilisation

Pour bien documenter chaque cas d'utilisation, on écrit une **fiche descriptive**.

1. Identification

- Nom du cas d'utilisation.
- Acteurs concernés.

- Objectif.

2. Description des scénarios

- **Scénario principal (flux normal)** : décrit les étapes standards.
- **Scénarios alternatifs (flux exceptionnels)** : décrivent ce qui se passe en cas d'erreur ou d'alternative.

  Exemple :

  - Cas *Payer commande*.
  - Scénario principal : le client choisit mode de paiement, la banque valide, commande confirmée.
  - Scénario alternatif : carte refusée → message d'erreur.

3. Exigences non fonctionnelles

- Contraintes qui ne concernent pas la fonctionnalité en elle-même mais son exécution.

  Exemples :

  - Performance (*le paiement doit s'effectuer en moins de 5 secondes*).
  - Sécurité (*les données bancaires doivent être chiffrées*).
  - Disponibilité (*le service doit être accessible 24h/24*).

OFPPT

Formateur : AARAB HAFID